Plateforme SOC intelligente

Guide d'implémentation étape par étape

Kali · pfSense · Suricata · ClamAV · Wazuh · Shuffle · Cortex · MISP · TheHive
API ML (XGBoost, HDBSCAN, Isolation Forest) · FastAPI (réponse) · SOC Copilot

Rapport technique — débutant en cybersécurité

Version finale — 19 octobre 2026 (alignée sur le README du projet)

Table des matières

1. Introduction et objectif **5**

1.1 Comment lire ce guide . . . 5

2. Vue d'ensemble de l'architecture **6**

2.1 Lecture du schéma . . . 6

2.2 Rôle de chaque composant . . . 7

2.3 Flux de données (à retenir) . . . 8

3. Prérequis, dimensionnement et plan réseau **9**

3.1 Hyperviseur . . . 9

3.2 Ressources recommandées . . . 9

3.3 Si votre PC est limité . . . 9

3.4 Plan d'adressage . . . 9

3.5 Préparation commune à toutes les VM Linux . . . 10

4. Phase 1 — pfSense (pare-feu) et segmentation **11**

4.1 Installation . . . 11

4.2 Règles de pare-feu minimales . . . 11

4.3 Installer et configurer Suricata . . . 11

5. Phase 2 — Serveur SOC et Wazuh **13**

5.1 Préparer le serveur SOC . . . 13

5.2 Installer Wazuh (manager + indexer + dashboard) . . . 13

5.3 Vérifications . . . 13

5.4 Activer la réception syslog (Suricata/pfSense) . . . 14

6. Phase 3 — Ubuntu cible (agent Wazuh + ClamAV) **15**

6.1 Installer l'agent Wazuh . . . 15

6.2 Activer les logs utiles . . . 15

6.3 Installer ClamAV . . . 16

7. Phase 4 — Windows cible (agent, Sysmon, WinRM) **17**

7.1 Installer l'agent Wazuh . . . 17

7.2 Installer Sysmon . . . 17

7.3 Activer les journaux Windows utiles . . . 17

7.4 Préparer WinRM (réponse active depuis FastAPI) . . . 18

8. Phase 5 — Kali Linux (attaquant) **19**

9. Phase 6 — Tester Wazuh avant d'ajouter le reste **20**

9.1 Scénarios de test et résultats attendus . . . 20

9.2 Créer une règle personnalisée (exemple) . . . 20

10. Phase 7 — TheHive et Cortex (gestion d'incidents et analyse) 22

10.1 Installer Docker sur le serveur SOC . . . 22
10.2 Déployer TheHive + Cortex . . . 22
10.3 Premier accès et configuration . . . 23
10.4 Test . . . 23

11. Phases 8 et 9 — MISP et VirusTotal optionnel (threat intelligence) 24

11.1 Installer MISP (Docker) . . . 24
11.2 Relier MISP à Cortex et TheHive . . . 24
11.3 Ajouter VirusTotal (optionnel) . . . 24
11.4 Test . . . 24

12. Phase 10 — Shuffle (SOAR / automatisation) 26

12.1 Installation . . . 26
12.2 Connecter Wazuh à Shuffle . . . 26
12.3 Workflow recommandé . . . 27

13. Phase 11 — Module Machine Learning et API ML 28

13.1 Construire le dataset . . . 28
13.2 Variables (features) à fabriquer . . . 29
13.3 Entraînement des modèles . . . 29
13.4 Inférence : score, SHAP, clustering, contre-mesure . . . 30
13.5 Exposer le module : l'API ML (échanges JSON avec Shuffle) . . . 31

14. Phase 12 — FastAPI (réponse automatisée) 33

14.1 Environnement Python . . . 33
14.2 Code de l'API de réponse . . . 34
14.3 Lancer et tester . . . 34
14.4 Lancer au démarrage (systemd) . . . 35

15. Phase 13 — SOC Copilot (LLM + RAG) 36

15.1 Principe du RAG . . . 36
15.2 Choix du LLM . . . 36
15.3 Squelette du Copilot . . . 36

16. Phase 14 — Scénarios de bout en bout et démonstration 38

16.1 Scénario A — Brute force SSH . . . 38
16.2 Scénario B — Fichier malveillant (EICAR) sur Ubuntu . . . 38
16.3 Scénario C — Abus de compte sur Windows (famille Active Directory) . . . 38
16.4 Scénario D — Scan réseau . . . 38
16.5 Indicateurs à mesurer pour le rapport . . . 38

17. Dépannage : problèmes fréquents 40

18. Sécurisation de la plateforme elle-même **41**

19. Planning proposé et ordre des étapes **42**

19.1 Les 4 questions à poser pour chaque outil 42

19.2 Bonnes habitudes 42

20. Glossaire **43**

21. Documentation officielle à consulter **44**

1. Introduction et objectif

Ce document décrit, étape par étape, comment construire un **laboratoire SOC complet** (Security Operations Center) à partir de l'architecture présentée dans le schéma du projet. Il est écrit pour un débutant : chaque phase explique *pourquoi* on installe un outil, *où* on l'installe, *ce qu'il doit recevoir* et *comment vérifier* que tout fonctionne.

Le but n'est pas seulement d'installer des logiciels, mais de construire un **pipeline de détection et de réponse** :

1. Une attaque est simulée depuis Kali Linux contre les machines cibles.
2. Les équipements de sécurité (pfSense/Suricata, ClamAV, agents Wazuh) produisent des logs et des alertes.
3. Wazuh centralise, corrèle et stocke ces événements (indexer + dashboard).
4. Shuffle (SOAR) orchestre l'enrichissement avec Cortex et MISP (VirusTotal reste optionnel, via un analyseur Cortex).
5. Shuffle interroge l'**API ML** (échanges JSON) : classification, score de risque, explicabilité, clustering, anomalies, contre-mesure. Une contre-mesure validée peut être déclenchée via **FastAPI** (HTTP POST puis commande WinRM sur Windows).
6. TheHive transforme l'alerte en incident (case) suivi par l'analyste.
7. Le SOC Copilot (LLM + RAG) aide l'analyste à comprendre les alertes. **L'analyste SOC prend la décision finale.**

Cadre légal et éthique
Toutes les attaques de ce guide sont réalisées **uniquement dans votre laboratoire isolé**, sur vos propres machines virtuelles. Ne lancez jamais de scans ou d'attaques sur un réseau ou un système qui ne vous appartient pas.

Conformité avec le README du projet
Cette version suit le flux du README : Shuffle reçoit les alertes Wazuh, les enrichit (Cortex, MISP), interroge l'**API ML** en JSON, puis crée l'incident dans TheHive. **FastAPI** sert uniquement à la réponse automatisée (HTTP POST puis WinRM). Le **score de risque** est encore « à définir » dans le README : ce guide propose une formule de travail, à valider. Les familles de classification sont : **attaque réseau, malware, Active Directory** (+ trafic bénin). VirusTotal n'est pas dans le README : il est présenté comme option.

1.1 Comment lire ce guide

- Chaque phase se termine par une **checklist de validation** : ne passez à la phase suivante que si tout est coché.
- Les commandes sont données pour **Ubuntu 22.04/24.04**, **Windows 10/11** et **Kali**. Les numéros de version des outils évoluent : vérifiez toujours la documentation officielle (indiquée en fin de document) pour les versions exactes.
- Prenez un **snapshot de VM** avant et après chaque phase importante.

2. Vue d'ensemble de l'architecture

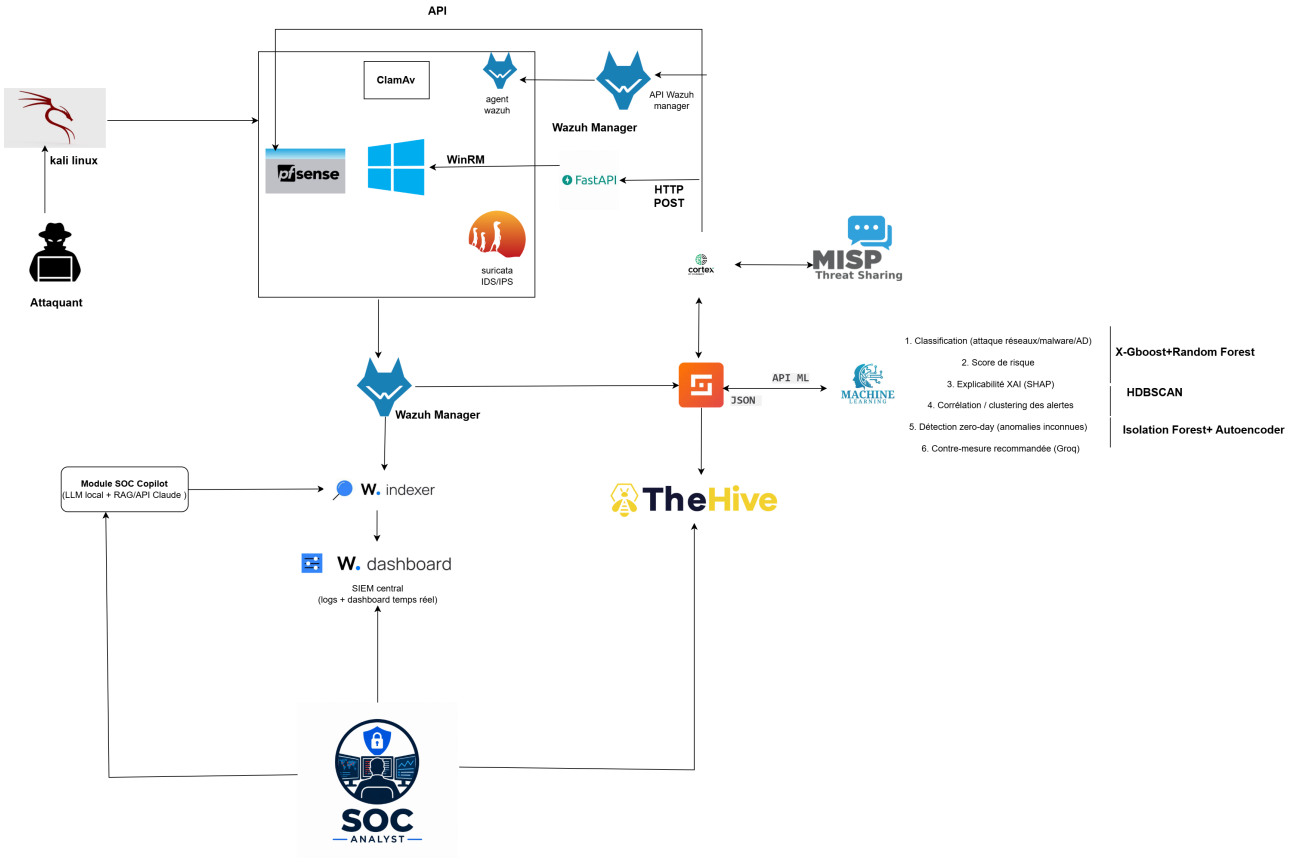


Figure 1 — Architecture cible du projet (schéma fourni).

2.1 Lecture du schéma

Le schéma se lit en quatre blocs :

- **Attaquant** : Kali Linux génère des attaques de test (scan, brute force, web).
- **Zone surveillée** (grand rectangle) : pfSense (pare-feu) avec Suricata (IDS/IPS), une machine Windows, un agent Wazuh et ClamAV. Le trafic de Kali passe par pfSense avant d'atteindre les cibles.
- **Cœur SOC** : Wazuh Manager (avec son API), Wazuh indexer, Wazuh dashboard (SIEM central), Shuffle, Cortex, MISP, TheHive.
- **Couche intelligence** : module ML exposé par une **API ML** (JSON, appelée par Shuffle), **FastAPI** de réponse automatisée (HTTP POST, puis WinRM vers Windows), SOC Copilot (LLM local + RAG / API Claude).

2.2 Rôle de chaque composant

Composant	Rôle dans le pipeline	Où l'installer
Kali Linux	Machine d'attaque : génère des événements de test	VM dédiée (réseau « attaquant »)
pfSense	Pare-feu/routeur entre attaquant et zone cible ; segmentation	VM avec 3 interfaces réseau
Suricata (IDS/IPS)	Détection réseau (signatures) ; produit des logs EVE JSON	Package pfSense (ou sur le SOC)
ClamAV	Antivirus open source : détecte fichiers malveillants (EICAR en test)	Sur Ubuntu cible (et/ou serveur)
Agent Wazuh	Collecte logs, intégrité des fichiers (FIM), inventaire, réponse active	Ubuntu cible + Windows cible
Wazuh Manager + API	Reçoit/analyse les événements, génère les alertes, expose une API (port 55000)	Serveur SOC
Wazuh indexer + dashboard	Stockage/recherche des alertes ; interface web SIEM	Serveur SOC
Shuffle (SOAR)	Orchestration : enrichissement, création de cases, appel ML	Serveur SOC (Docker)
Cortex	Analyseurs automatiques (IP, hash, domaine, URL)	Serveur SOC (Docker)
MISP	Plateforme de threat intelligence (IOC, campagnes)	Serveur SOC (Docker ou VM)
VirusTotal (optionnel)	Enrichissement complémentaire (non prévu dans le README) : réputation fichiers/IP/domaines via un analyseur Cortex	Service externe (clé API gratuite)
TheHive	Gestion d'incidents : cases, tâches, observables, historique	Serveur SOC (Docker)
FastAPI	API de réponse automatisée : reçoit un HTTP POST validé et exécute la contre-mesure par WinRM	Serveur SOC (port 8000)
API ML (module ML)	Classification, score de risque, SHAP, clustering, anomalies, contre-mesure (Groq) ; échanges JSON avec Shuffle	Serveur SOC (Python, port 8001)
SOC Copilot	Assistant LLM + RAG sur les alertes indexées	Serveur SOC (Python)

2.3 Flux de données (à retenir)

#	Flux	Protocole / port
1	Kali → pfSense → cibles (attaques de test)	Tous (scan, SSH, HTTP…)
2	Agents Wazuh → Wazuh Manager	TCP 1514 (événements), TCP 1515 (enrôlement)
3	Suricata (pfSense) → Wazuh	Syslog UDP 514 (EVE JSON)
4	Wazuh Manager → indexer → dashboard	9200 (indexer), 443 (dashboard)
5	Wazuh Manager → Shuffle (webhook, niveau ≥ 7)	HTTP(S) vers Shuffle (3001/5001)
6	Shuffle ↔ Cortex ↔ MISP (VirusTotal optionnel via Cortex)	HTTP 9001 (Cortex), 8443 (MISP)
7	Shuffle ↔ API ML (JSON : alerte + enrichissement → catégorie, score, SHAP…)	HTTP POST 8001
8	Shuffle → TheHive (création du case pour l'analyste)	HTTP 9000
9	Shuffle → FastAPI (HTTP POST) puis FastAPI → Windows (commande WinRM)	HTTP 8000, puis WinRM 5985/5986
10	SOC Copilot → indexer ; analyste → dashboard, TheHive, Copilot	HTTPS

3. Prérequis, dimensionnement et plan réseau

3.1 Hyperviseur

Choix possibles : **VirtualBox** (simple, gratuit), **VMware Workstation** (plus stable) ou **Proxmox** (si vous avez un serveur dédié). Pour un débutant, VirtualBox suffit. Si vous avez moins de 32 Go de RAM, activez les composants progressivement (voir 3.3).

3.2 Ressources recommandées

VM	vCPU	RAM	Disque	OS
Serveur SOC (Wazuh, TheHive, Cortex, MISP, Shuffle, API ML, FastAPI)	6–8	16–24 Go (min 12)	150 Go	Ubuntu Server 22.04/24.04 LTS
pfSense + Suricata	2	2–4 Go	20 Go	pfSense CE
Ubuntu cible (agent Wazuh + ClamAV)	2	2 Go	25 Go	Ubuntu 22.04/24.04
Windows cible (agent + Sysmon)	2–4	4–8 Go	60 Go	Windows 10/11 (évaluation)
Kali Linux	2	2–4 Go	30 Go	Kali (dernière version)

3.3 Si votre PC est limité

- Phase A (≈ 12 Go) : SOC (Wazuh seul) + Ubuntu cible + Kali.
- Phase B : ajoutez Windows + Sysmon, puis pfSense/Suricata.
- Phase C : ajoutez TheHive + Cortex, puis MISP, puis Shuffle.
- Phase D : API ML + FastAPI + Copilot (peuvent tourner sur la machine hôte si le SOC manque de RAM).

3.4 Plan d'adressage

Avec pfSense, on utilise trois segments. Si vous commencez sans pfSense, mettez tout dans 192.168.56.0/24 (host-only) puis migrez.

Segment	Réseau (VirtualBox)	Machines	IP
WAN / Internet	NAT	pfSense (WAN)	DHCP
ATTAQUANT (OPT1)	Réseau interne « net-attaque »	Kali pfSense OPT1	10.10.10.20 10.10.10.1
CIBLES (LAN)	Réseau interne « net-cibles »	pfSense LAN Ubuntu cible Windows cible	192.168.56.1 192.168.56.30 192.168.56.40
SOC (OPT2)	Réseau interne « net-soc »	pfSense OPT2 Serveur SOC	192.168.57.1 192.168.57.10

Astuce
Ajoutez sur chaque VM (sauf pfSense) une carte NAT temporaire pour installer les paquets, puis désactivez-la pour garder le labo isolé. Pour les cibles, il faudra autoriser dans pfSense les flux vers le SOC : 1514/1515 TCP (agents) et 514 UDP (syslog).

3.5 Préparation commune à toutes les VM Linux

Commandes de base (Ubuntu)

```
sudo apt update && sudo apt -y upgrade
sudo apt -y install curl wget gnupg unzip net-tools vim chrony openssh-server
sudo timedatectl set-timezone Europe/Rome
sudo systemctl enable --now chrony ssh
```

Synchronisation de l'heure
Toutes les VM doivent avoir la **même heure** (NTP). Un décalage d'horloge fausse la corrélation des alertes et casse certaines authentifications (certificats, tokens).

Checklist de validation

- ☐ L'hyperviseur est installé et les 3 réseaux internes sont créés.
- ☐ Les VM sont créées avec les ressources indiquées (ou version allégée par phases).
- ☐ Un tableau d'adresses IP est noté dans votre carnet de bord.
- ☐ Un snapshot « vm-propre » est pris pour chaque VM.

4. Phase 1 — pfSense (pare-feu) et segmentation

Pourquoi ? Dans le schéma, tout le trafic de l'attaquant traverse pfSense. Cela permet d'appliquer des règles de pare-feu, de faire tourner Suricata sur le trafic, et de simuler un vrai réseau d'entreprise.

4.1 Installation

1. Téléchargez l'ISO pfSense CE (site netgate.com) et créez une VM FreeBSD 64 bits avec **4 cartes réseau** : WAN (NAT), LAN (net-cibles), OPT1 (net-attaque), OPT2 (net-soc).
2. Installez avec les options par défaut (ZFS ou UFS, selon votre préférence).
3. À la console, assignez les interfaces : WAN = em0, LAN = em1, OPT1 = em2, OPT2 = em3 (les noms peuvent varier).
4. Réglez les IP : LAN 192.168.56.1/24, OPT1 10.10.10.1/24, OPT2 192.168.57.1/24 (option 2 du menu console).
5. Depuis une VM du LAN, ouvrez **https://192.168.56.1** (identifiants par défaut : admin / pfsense — **changez-les immédiatement**).

4.2 Règles de pare-feu minimales

Interface	Action	Source → Destination	Port	But
OPT1 (attaque)	Pass	10.10.10.0/24 → 192.168.56.0/24	Tous	Autoriser Kali à attaquer les cibles
OPT1 (attaque)	Block	10.10.10.0/24 → 192.168.57.0/24	Tous	Protéger le SOC contre l'attaquant
LAN (cibles)	Pass	192.168.56.0/24 → 192.168.57.10	TCP 1514, 1515	Agents Wazuh → Manager
LAN (cibles)	Pass	pfSense → 192.168.57.10	UDP 514	Syslog Suricata → Wazuh
OPT2 (SOC)	Pass	192.168.57.10 → 192.168.56.0/24	TCP 5985/5986	WinRM depuis FastAPI
WAN	—	—	—	Aucune règle entrante (défaut = bloqué)

4.3 Installer et configurer Suricata

1. *System → Package Manager → Available Packages* : installez **suricata**.
2. *Services → Suricata → Global Settings* : activez les règles **ET Open** (gratuites) et planifiez la mise à jour.
3. *Interfaces → Add* : choisissez l'interface **LAN** (là où le trafic vers les cibles est visible) ; cochez **EVE JSON Log** (types : alert, http, dns, tls, files).
4. Démarrez en mode **IDS** (alerte seulement). Passez en IPS (blocage) uniquement quand vous maîtrisez les faux positifs.
5. Activez l'envoi syslog : *Status → System Logs → Settings → Remote Logging* : serveur 192.168.57.10:514 (contenu : Suricata/EVE).

Pourquoi EVE JSON ?
EVE est le format JSON de Suricata. Wazuh sait le décoder nativement (groupe de règles « suricata »), ce qui donne des alertes riches : signature, IP source/destination, catégorie.

Checklist de validation

- ☐ Les VM Kali, cibles et SOC se joignent selon les règles prévues (Kali ne peut pas atteindre le SOC).
- ☐ L'interface web pfSense est accessible avec un mot de passe modifié.
- ☐ Suricata est démarré sur le LAN et des alertes apparaissent dans *Services* → *Suricata* → *Alerts* après un scan Nmap de Kali.

5. Phase 2 — Serveur SOC et Wazuh

Pourquoi Wazuh en premier ? Wazuh est le cœur du SIEM : il collecte les logs, génère les alertes et centralise la sécurité. Tous les autres outils (Shuffle, TheHive, ML, Copilot) consomment ses alertes.

5.1 Préparer le serveur SOC

1. Installez Ubuntu Server 22.04/24.04 LTS (OpenSSH coché), IP fixe 192.168.57.10, passerelle 192.168.57.1.
2. Appliquez les commandes de base (section 3.5).
3. Réglez l'IP fixe avec netplan (exemple ci-dessous).

Netplan

```
# /etc/netplan/00-soc.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses: [192.168.57.10/24]
      routes:
        - to: default
          via: 192.168.57.1
      nameservers:
        addresses: [1.1.1.1, 9.9.9.9]
```

Appliquer

```
sudo netplan apply && ip a && ping -c 2 192.168.57.1
```

5.2 Installer Wazuh (manager + indexer + dashboard)

Pour un laboratoire, on utilise l'**assistant d'installation tout-en-un** de Wazuh (les trois composants sur la même machine). Vérifiez le numéro de version courant sur la documentation officielle avant de lancer.

Installation tout-en-un

```
# Téléchargement de l'assistant (remplacer 4.x par la version courante)
curl -sO https://packages.wazuh.com/4.x/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
# À la fin, NOTEZ le mot de passe de l'utilisateur 'admin' affiché.
```

À la fin de l'installation, le script affiche l'URL du dashboard (https://192.168.57.10) et le mot de passe admin. Si vous l'avez perdu, il est dans l'archive `wazuh-install-files.tar` créée dans le répertoire courant.

5.3 Vérifications

Services et ports

```
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard
sudo ss -tulpen | grep -E '1514|1515|55000|9200|443'
sudo tail -n 30 /var/ossec/logs/ossec.log
```

Composant	Port	Rôle
Wazuh dashboard	443/TCP	Interface web SIEM
Wazuh indexer	9200/TCP	Stockage / recherche (API OpenSearch)

Composant	Port	Rôle
Wazuh manager (agents)	1514/TCP	Réception des événements des agents
Wazuh manager (enrôlement)	1515/TCP	Enregistrement des nouveaux agents
Wazuh API	55000/TCP	API REST (utilisée par FastAPI/Shuffle)
Syslog (à activer)	514/UDP	Réception Suricata/pfSense

5.4 Activer la réception syslog (Suricata/pfSense)

Éditez `/var/ossec/etc/ossec.conf` et ajoutez un bloc `<remote>` :

ossec.conf

```
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.56.1</allowed-ips>   <!-- IP LAN de pfSense -->
</remote>
```

Appliquer

```
sudo systemctl restart wazuh-manager
```

Bonne pratique
Gardez une copie de sauvegarde avant chaque modification : `sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak`. Après un changement, contrôlez la syntaxe avec `sudo /var/ossec/bin/wazuh-analysisd -t`.

Checklist de validation

- ☐ Le dashboard s'ouvre en HTTPS et vous vous connectez avec admin.
- ☐ Les trois services sont *active (running)*.
- ☐ Les ports 1514, 1515, 55000, 9200 et 443 sont en écoute.
- ☐ Snapshot « soc-wazuh-ok » pris.

6. Phase 3 — Ubuntu cible (agent Wazuh + ClamAV)

Pourquoi ? Cette machine représente un poste/serveur Linux d'entreprise surveillé. L'agent remonte les logs d'authentification, les modifications de fichiers et l'activité système ; ClamAV détecte les fichiers malveillants.

6.1 Installer l'agent Wazuh

Agent Wazuh (Ubuntu)

```
# Dépôt Wazuh
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --no-default-keyring \
  --keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import && chmod 644
      /usr/share/keyrings/wazuh.gpg
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" \
  | sudo tee /etc/apt/sources.list.d/wazuh.list
sudo apt update

# Installation + enrôlement auprès du manager
sudo WAZUH_MANAGER="192.168.57.10" WAZUH_AGENT_NAME="ubuntu-cible" apt -y install wazuh-agent
sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent
```

Vérifiez dans le dashboard : *Agents* doit afficher **ubuntu-cible** avec le statut *Active*. Sinon, regardez `/var/ossec/logs/ossec.log` sur l'agent.

6.2 Activer les logs utiles

- **Authentification** : `/var/log/auth.log` (SSH, sudo, création d'utilisateurs) — collecté par défaut.
- **FIM (intégrité des fichiers)** : surveillez `/etc`, `/home`, `/var/www` en temps réel.
- **Commandes (auditd)** : optionnel mais très utile pour voir les commandes exécutées.
- **Journaux système** : syslog, dpkg (installation de paquets).

Surveillance FIM + logs

```
<!-- /var/ossec/etc/ossec.conf (agent) -->
<syscheck>
  <directories realtime="yes" check_all="yes">/etc,/home,/var/www,/tmp</directories>
</syscheck>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/auth.log</location>
</localfile>
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/clamav/scan.log</location>
</localfile>
```

Appliquer

```
sudo systemctl restart wazuh-agent
```

6.3 Installer ClamAV

ClamAV + scan planifié

```
sudo apt -y install clamav clamav-daemon
sudo systemctl stop clamav-freshclam && sudo freshclam    # mise à jour des signatures
sudo systemctl enable --now clamav-freshclam clamav-daemon
sudo mkdir -p /var/log/clamav && sudo touch /var/log/clamav/scan.log

# Scan planifié (cron, toutes les heures) avec journal lisible par Wazuh
echo '0 * * * * root clamscan -r -i --log=/var/log/clamav/scan.log /home /tmp /var/www' \
  | sudo tee /etc/cron.d/clamscan
```

Test sûr : le fichier **EICAR** est un faux « virus » standard, inoffensif, reconnu par tous les antivirus.

Test EICAR

```
cd /tmp && echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' > eicar.txt
clamscan -i /tmp/eicar.txt          # doit afficher: Eicar-Signature FOUND
```

Checklist de validation

- ☐ L'agent « ubuntu-cible » est Active dans Wazuh.
- ☐ Créer un fichier dans /etc ou /tmp produit une alerte FIM (syscheck) dans le dashboard.
- ☐ Le test EICAR est détecté par ClamAV et la ligne du log apparaît dans Wazuh.

7. Phase 4 — Windows cible (agent, Sysmon, WinRM)

Pourquoi ? Windows produit énormément d'événements utiles pour un SOC (authentifications, processus, PowerShell). Sysmon enrichit ces logs ; WinRM permet à FastAPI d'exécuter des actions de réponse.

7.1 Installer l'agent Wazuh

Agent Wazuh (Windows)

```
# PowerShell en administrateur (remplacer 4.x.x par la version courante)
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.x.x-1.msi -OutFile
    $env:TEMP\wazuh-agent.msi
msiexec.exe /i $env:TEMP\wazuh-agent.msi /q WAZUH_MANAGER="192.168.57.10" WAZUH_AGENT_NAME="windows-
    cible"
NET START WazuhSvc
```

7.2 Installer Sysmon

Sysmon (Microsoft Sysinternals) journalise le lancement de processus, les connexions réseau, la création de fichiers, etc. Utilisez une configuration communautaire reconnue (par exemple « sysmon-config » de SwiftOnSecurity ou celle de Olaf Hartong).

Sysmon

```
# Télécharger Sysmon (Sysinternals) et une configuration XML, puis :
.\Sysmon64.exe -accepteula -i sysmonconfig.xml
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5
```

Dites ensuite à l'agent Wazuh de lire le canal Sysmon. Éditez `C:\Program Files (x86)\ossec-agent\ossec.conf` :

ossec.conf (Windows)

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
<localfile>
  <location>Microsoft-Windows-PowerShell/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Redémarrer l'agent

```
Restart-Service -Name wazuh
```

7.3 Activer les journaux Windows utiles

- **Security** : stratégie d'audit — ouverture de session, création de comptes, changements de privilèges (*secpol.msc* → *Stratégie d'audit avancée*).
- **PowerShell** : activer *Script Block Logging* et *Module Logging* (stratégie de groupe locale : *gpedit.msc*).
- **System / Application** : collectés par défaut par l'agent.

7.4 Préparer WinRM (réponse active depuis FastAPI)

WinRM (laboratoire)

```
# PowerShell administrateur sur la cible Windows
Enable-PSRemoting -Force
Set-Item WSMan:\localhost\Service\Auth\Basic -Value $true          # LABO UNIQUEMENT
Set-Item WSMan:\localhost\Service\AllowUnencrypted -Value $true     # LABO UNIQUEMENT
New-NetFirewallRule -DisplayName "WinRM-SOC" -Direction Inbound -Protocol TCP `
  -LocalPort 5985 -RemoteAddress 192.168.57.10 -Action Allow
# Compte dédié, non-administrateur de domaine, avec droits limités
net user svc_soc "MotDePasseFort!2026" /add
```

Sécurité WinRM
Basic + non chiffré est acceptable **uniquement dans un labo isolé**. En conditions réelles : WinRM sur HTTPS (5986) avec certificat, authentification Kerberos/NTLM, compte dédié à privilèges minimaux, et liste blanche de commandes côté FastAPI.

Checklist de validation

- ☐ L'agent « windows-cible » est Active dans Wazuh.
- ☐ Lancer PowerShell ou créer un compte test produit des événements visibles dans le dashboard.
- ☐ Les événements Sysmon (ID 1 = process, ID 3 = réseau) apparaissent dans Wazuh.
- ☐ Depuis le SOC : `python -c "import winrm"` après installation de pywinrm, et un test de commande simple réussit.

8. Phase 5 — Kali Linux (attaquant)

Pourquoi ? Kali ne sert pas à « attaquer vraiment » : elle génère des **événements de test reproductibles** pour vérifier que le SOC détecte correctement, et pour produire des données d'entraînement du module ML.

1. Installez Kali dans une VM reliée au réseau **net-attaque** (IP 10.10.10.20, passerelle 10.10.10.1).
2. Mettez à jour : `sudo apt update && sudo apt -y full-upgrade`.
3. Vérifiez les outils : nmap, hydra, nikto, gobuster, sqlmap, metasploit-framework, nuclei.
4. Testez la connectivité : `ping 192.168.56.30` (doit répondre) et `ping 192.168.57.10` (doit être bloqué).

Checklist de validation

- ☐ Kali joint les cibles mais pas le serveur SOC.
- ☐ Nmap fonctionne depuis Kali vers 192.168.56.30 et 192.168.56.40.

9. Phase 6 — Tester Wazuh avant d'ajouter le reste

Avant TheHive, MISP ou Shuffle, validez que **Wazuh seul** détecte bien. C'est la base de tout le pipeline.

9.1 Scénarios de test et résultats attendus

Où	Action de test	Ce que Wazuh doit montrer
Ubuntu cible	3–5 échecs SSH depuis Kali : `ssh fakeuser@192.168.56.30`	Règles « authentication failed » (5710, 5716), puis brute force (5712) si répété
Ubuntu cible	`sudo touch /etc/test-fim.conf` puis modification	Alertes syscheck : fichier ajouté/modifié (rule 550/554)
Ubuntu cible	`sudo adduser testsoc`	Création d'utilisateur (rules 5902, 5901)
Ubuntu cible	`sudo apt install -y cowsay`	Événement d'installation de paquet (dpkg)
Ubuntu cible	Fichier EICAR + scan ClamAV	Alerte issue du log ClamAV
Windows	Ouverture de session, échec de mot de passe	Événements 4624 / 4625 (rules 60106 / 60122)
Windows	`powershell -enc ...` (commande encodée inoffensive)	Sysmon ID 1 + alerte PowerShell suspect
Windows	`net user testsoc P@ss123! /add`	Événement 4720 : compte créé
Kali → cibles	`nmap -sS -sV -p- 192.168.56.30`	Alertes Suricata (ET SCAN) via syslog dans Wazuh
Kali → cibles	`hydra -l root -P rockyou.txt ssh://192.168.56.30 -t 4` (limiter!)	Brute force SSH, niveau d'alerte élevé
Kali → cibles	`nikto -h http://192.168.56.30` (si un serveur web est installé)	Alertes web (rules 31xxx) + Suricata

Où regarder dans le dashboard
Threat Hunting / *Security events* : filtrez par agent, par `rule.level` (≥ 7 = important) et par `rule.groups` (authentication_failed, syscheck, suricata, windows…). Notez pour chaque test : l'heure, la règle, le niveau, l'agent.

9.2 Créer une règle personnalisée (exemple)

Une bonne pratique SOC : adapter les règles à votre contexte. Exemple : élever le niveau d'une alerte de création de compte.

Règle locale

```
<!-- /var/ossec/etc/rules/local_rules.xml -->
<group name="local,custom,">
  <rule id="100100" level="10">
    <if_sid>60109</if_sid>                    <!-- compte utilisateur créé (Windows) -->
    <description>Compte local créé sur un poste surveillé</description>
    <mitre><id>T1136.001</id></mitre>
  </rule>
</group>
```

Tester puis appliquer

```
sudo /var/ossec/bin/wazuh-logtest   # tester une ligne de log avant de redémarrer
sudo systemctl restart wazuh-manager
```

Checklist de validation

- ☐ Chaque scénario produit au moins une alerte visible, avec le bon agent et un niveau de sévérité.
- ☐ Les alertes Suricata de Kali arrivent dans Wazuh.
- ☐ Vous avez un fichier « journal-des-tests » avec date, action, alerte obtenue, captures d'écran.

10. Phase 7 — TheHive et Cortex (gestion d'incidents et analyse)

TheHive transforme une alerte en **incident suivi** (case, tâches, observables, historique). **Cortex** exécute des **analyseurs** automatiques sur les observables (IP, hash, domaine, URL) et renvoie un verdict. Les deux fonctionnent très bien avec Docker.

10.1 Installer Docker sur le serveur SOC

Docker + paramètre noyau pour Elasticsearch/OpenSearch

```
sudo apt -y install ca-certificates curl
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER && newgrp docker
docker --version && docker compose version
sudo sysctl -w vm.max_map_count=262144
echo 'vm.max_map_count=262144' | sudo tee /etc/sysctl.d/99-es.conf
```

Conflits de ports sur le serveur SOC
Wazuh utilise déjà **443** (dashboard) et **9200** (indexer). N'exposez donc **pas** l'Elasticsearch de TheHive/Cortex sur 9200 (laissez-le interne au réseau Docker) et publiez MISP sur **8443**. Plan de ports proposé : TheHive 9000, Cortex 9001, MISP 8443, Shuffle 3001, FastAPI (réponse) 8000, API ML 8001.

10.2 Déployer TheHive + Cortex

TheHive 5 a besoin d'une base (Cassandra) et d'un moteur d'indexation (Elasticsearch). Voici un squelette Docker Compose **à adapter** aux versions recommandées par la documentation StrangeBee (images et variables évoluent).

docker-compose.yml (TheHive + Cortex)

```
# ~/soc/thehive/docker-compose.yml  (squelette — vérifier versions/variables dans la doc)
services:
  cassandra:
    image: cassandra:4
    environment: { MAX_HEAP_SIZE: 1G, HEAP_NEWSIZE: 400M }
    volumes: [ cassandra_data:/var/lib/cassandra ]
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:7.17.x
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - "ES_JAVA_OPTS=-Xms1g -Xmx1g"
    volumes: [ es_data:/usr/share/elasticsearch/data ]
  thehive:
    image: strangebee/thehive:5.x
    depends_on: [cassandra, elasticsearch]
    ports: ["9000:9000"]
    command:
      - --secret
      - "CHANGE_ME_secret_32_chars_min"
      - --cql-hostnames
      - cassandra
      - --index-backend
      - elasticsearch
      - --es-hostnames
      - elasticsearch
  cortex:
    image: thehiveproject/cortex:3.x
    depends_on: [elasticsearch]
    ports: ["9001:9001"]
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock   # nécessaire pour les analyseurs Docker
volumes: { cassandra_data: {}, es_data: {} }
```

Lancer

```
cd ~/soc/thehive && docker compose up -d && docker compose ps && docker compose logs -f thehive
```

10.3 Premier accès et configuration

1. TheHive : **http://192.168.57.10:9000** — compte initial par défaut (admin@thehive.local / secret) : **changez-le immédiatement**.
2. Créez une **organisation** « SOC-Lab », puis un utilisateur analyste et un utilisateur « service » (type API) pour Shuffle ; générez sa **clé API**.
3. Cortex : **http://192.168.57.10:9001** — créez le superadmin, une organisation « SOC-Lab », un utilisateur *orgAdmin* et un utilisateur *read,analyze* pour TheHive/Shuffle (avec clé API).
4. Dans Cortex, onglet *Organization* → *Analyzers* : activez **VirusTotal_GetReport**, **AbuseIPDB**, **MISP_2_1** (recherche dans MISP), **Abuse_Finder** et **Shodan** si vous avez des clés.
5. Relier TheHive à Cortex : fichier de configuration TheHive (section *connectors* → *cortex*) avec l'URL et la clé API de Cortex.

10.4 Test

1. Dans TheHive, créez un **case de test** avec un observable IP (ex. 8.8.8.8).
2. Lancez un analyseur depuis l'observable : le rapport doit apparaître.
3. Dans Cortex, vérifiez dans *Jobs* que l'analyse s'est exécutée.

Checklist de validation

- ☐ TheHive et Cortex sont accessibles, mots de passe par défaut changés.
- ☐ Une clé API TheHive (pour Shuffle/FastAPI) et une clé API Cortex sont notées dans un coffre (gestionnaire de mots de passe).
- ☐ Un analyseur Cortex retourne un rapport sur une IP de test.

11. Phases 8 et 9 — MISP et VirusTotal optionnel (threat intelligence)

MISP est votre base de renseignement sur les menaces : IOC (IP, domaines, hashes), événements, campagnes, tags. **VirusTotal** (optionnel, absent du README) ajoute la réputation d'un fichier/IP/domaine selon des dizaines de moteurs. Comparer MISP et VirusTotal via Cortex rend le verdict plus fiable.

11.1 Installer MISP (Docker)

MISP via misp-docker

```
git clone https://github.com/MISP/misp-docker.git ~/soc/misp && cd ~/soc/misp
cp template.env .env
# Éditer .env : BASE_URL=https://192.168.57.10:8443 ; ADMIN_EMAIL ; ADMIN_PASSWORD ; mots de passe
    MySQL
# Dans docker-compose.yml, publier le port 8443:443 (le 443 est pris par Wazuh)
docker compose pull && docker compose up -d
docker compose logs -f misp-core
```

1. Connectez-vous à **https://192.168.57.10:8443** (certificat auto-signé : accepter l'avertissement).
2. Changez le mot de passe admin par défaut ; créez une organisation et un utilisateur API.
3. *Sync Actions* → *List Feeds* : activez des feeds gratuits (CIRCL OSINT, Botvrij.eu, abuse.ch…) puis cliquez *Fetch and store all feed data*.
4. *Administration* → *Auth Keys* : créez une **clé API** pour Cortex/Shuffle.

11.2 Relier MISP à Cortex et TheHive

- Cortex : analyseur **MISP_2_1** avec URL, clé API et option « cert_check = false » en labo.
- TheHive : possibilité d'exporter/importer des événements vers MISP (connecteur MISP), utile pour partager des IOC validés.

11.3 Ajouter VirusTotal (optionnel)

1. Créez un compte gratuit sur virustotal.com et copiez votre **clé API**.
2. Dans Cortex, configurez l'analyseur **VirusTotal_GetReport_3_1** (clé dans la configuration de l'analyseur).
3. Respectez les limites de l'offre gratuite (environ 4 requêtes/minute et un quota journalier) : prévoyez une pause ou un cache dans le workflow Shuffle.

> **Confidentialité**
> N'envoyez **jamais** à VirusTotal un fichier contenant des données sensibles : un fichier envoyé devient accessible à la communauté. Privilégiez la recherche par **hash** (SHA256) plutôt que l'envoi du fichier.

11.4 Test

Interroger MISP

```
# Recherche d'un IOC dans MISP via l'API
curl -sk -H "Authorization: MISP_API_KEY" -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  -d '{"returnFormat":"json","value":"8.8.8.8"}' \
  https://192.168.57.10:8443/attributes/restSearch | head -c 600
```

Test EICAR par hash : SHA256 `275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f` (connu de VirusTotal comme le fichier de test EICAR).

Checklist de validation

- ☐ MISP est accessible sur 8443 et contient des données de feeds.
- ☐ L'analyseur MISP_2_1 de Cortex répond.
- ☐ L'analyseur VirusTotal de Cortex retourne un verdict sur le hash EICAR.

12. Phase 10 — Shuffle (SOAR / automatisation)

Shuffle orchestre le pipeline : il reçoit l'alerte Wazuh, extrait les observables, appelle Cortex/MISP/VirusTotal, interroge FastAPI (ML) et crée le case dans TheHive. C'est l'icône orange placée entre Wazuh Manager, Cortex, le module ML et TheHive dans le schéma.

12.1 Installation

Shuffle (Docker Compose officiel)

```
git clone https://github.com/Shuffle/Shuffle.git ~/soc/shuffle && cd ~/soc/shuffle
mkdir -p shuffle-database && sudo chown -R 1000:1000 shuffle-database
docker compose up -d
# Interface : http://192.168.57.10:3001  (création du compte admin au premier accès)
```

12.2 Connecter Wazuh à Shuffle

Wazuh possède une intégration **shuffle** native. Dans Shuffle, créez un workflow avec un déclencheur **Webhook** et copiez son URL. Puis, sur le serveur Wazuh :

Intégration Wazuh → Shuffle

```
<!-- /var/ossec/etc/ossec.conf -->
<integration>
  <name>shuffle</name>
  <hook_url>http://192.168.57.10:3001/api/v1/hooks/webhook_XXXXXXXX</hook_url>
  <level>7</level>                        <!-- n'envoyer que les alertes importantes -->
  <alert_format>json</alert_format>
</integration>
```

Appliquer

```
sudo systemctl restart wazuh-manager
```

12.3 Workflow recommandé

Étape	Nœud Shuffle	Action
1	Webhook	Reçoit l'alerte Wazuh (JSON : rule, agent, data.srcip, syscheck.sha256_after…)
2	Shuffle Tools — Repeat back / Python	Extrait les observables : IP source, hash, nom d'hôte, utilisateur
3	Cortex	Lance les analyseurs sur l'IP / le hash (MISP, AbuseIPDB, VirusTotal si activé) et attend le rapport
4	MISP	Recherche l'IOC (attributes/restSearch) ; récupère événements et tags
5	HTTP	POST JSON vers l'**API ML** `http://192.168.57.10:8001/analyze` avec alerte + enrichissements
6	Condition	Selon le score de risque (seuil « à définir ») → création du case TheHive ; sinon alerte simple
7	TheHive	Création alerte/case avec sévérité, observables, résumé ML + SHAP + contre-mesure
8	User Input / Email	Demande de validation humaine avant toute action de réponse
9	HTTP (après validation)	POST vers **FastAPI** `http://192.168.57.10:8000/respond` → commande WinRM sur Windows

Principe « humain dans la boucle »
L'automatisation enrichit et propose ; une action sensible (isolement d'une machine, blocage d'une IP, désactivation d'un compte) doit rester **validée par l'analyste** au début. Vous pourrez automatiser davantage quand vous aurez mesuré la fiabilité du modèle.

Checklist de validation

- ☐ Une alerte Wazuh de niveau ≥ 7 déclenche le workflow (visible dans l'historique d'exécution Shuffle).
- ☐ Chaque nœud s'exécute sans erreur sur un cas réel (ex. brute force SSH depuis Kali).
- ☐ Un case apparaît dans TheHive avec les observables.

13. Phase 11 — Module Machine Learning et API ML

C'est la partie « intelligente ». Le schéma prévoit six fonctions, chacune associée à une famille d'algorithmes :

#	Fonction (schéma)	Algorithme	Sortie attendue
1	Classification (attaque réseau, malware, Active Directory)	XGBoost + Random Forest	Catégorie d'attaque + probabilités
2	Score de risque	**À définir** (le README ne fixe pas la méthode) ; formule de travail proposée en 13.4	Score 0–100
3	Explicabilité (XAI)	SHAP	Les 3–5 variables qui expliquent la décision
4	Corrélation / clustering des alertes	HDBSCAN	Identifiant de cluster (alertes liées à un même incident)
5	Détection zero-day (anomalies inconnues)	Isolation Forest + Autoencoder	Score d'anomalie
6	Contre-mesure recommandée	LLM via Groq (prompt contrôlé)	Texte court d'actions proposées

Score de risque : « à définir »
Le README indique « [à définir] » pour cette fonction. La formule donnée en 13.4 (probabilité du classifieur, niveau de la règle Wazuh, criticité de l'hôte, résultats MISP) est une **proposition de travail** : l'équipe doit la valider, la justifier et la calibrer sur ses propres données.

Conseil de débutant : n'entraînez pas le modèle trop tôt
Ordre correct : **1)** collecter les logs, **2)** comprendre les alertes, **3)** construire un jeu de données étiqueté, **4)** seulement ensuite entraîner. Un modèle sans données réalistes ne vaut rien.

Environnement Python du module ML (dossier ~/soc/ml, séparé de FastAPI)

```
sudo apt -y install python3-venv python3-pip
mkdir -p ~/soc/ml && cd ~/soc/ml && python3 -m venv .venv && source .venv/bin/activate
pip install fastapi "uvicorn[standard]" pydantic requests joblib scikit-learn xgboost shap hdbscan
    pandas numpy
```

13.1 Construire le dataset

1. Exécutez chaque scénario d'attaque de la section 9 plusieurs fois (ex. 20 fois chacun), en notant **l'heure de début/fin** et le **type** (étiquette).
2. Exportez les alertes Wazuh de ces fenêtres de temps depuis l'indexer.
3. Étiquetez selon les trois familles du README : *attaque_reseau* (scan, brute force, attaque web), *malware* (fichiers malveillants, EICAR) et *active_directory* (abus de comptes, authentifications anormales, élévation de privilèges). Générez aussi du trafic **normal** (navigation, mises à jour, connexions légitimes) pour la classe « benign ».
4. Complétez au besoin avec un jeu public (CICIDS2017, UNSW-NB15) pour les attaques réseau, en gardant à l'esprit que leur format diffère des alertes Wazuh.

Export depuis l'indexer (labo : verify=False)

```
# export_alerts.py — extraire les alertes de l'indexer Wazuh
import requests, json, pandas as pd
URL = "https://192.168.57.10:9200/wazuh-alerts-*/_search"
q = {"size": 10000, "query": {"range": {"timestamp": {"gte": "now-7d"}}}, "sort": [{"timestamp":
    "asc"}]}
r = requests.get(URL, auth=("admin", "MOT_DE_PASSE"), json=q, verify=False)
rows = [h["_source"] for h in r.json()["hits"]["hits"]]
pd.json_normalize(rows).to_csv("alerts_raw.csv", index=False)
print(len(rows), "alertes exportées")
```

13.2 Variables (features) à fabriquer

Variable	Source	Intérêt
rule_level, rule_id, nb de groupes	rule.level / rule.id / rule.groups	Gravité et nature de l'événement
firedtimes, fréquence par IP source (fenêtre 5 min)	rule.firedtimes, agrégation	Détecter brute force / scan
heure, jour, hors-heures (bool)	timestamp	Activités inhabituelles
type d'agent (linux/windows/réseau)	agent.name, decoder.name	Contexte d'hôte
nb de ports/hosts distincts ciblés	Suricata / Wazuh	Scan réseau
technique MITRE (encodée)	rule.mitre.id	Famille d'attaque
réputation IP/hash (VT, MISP)	Enrichissement Shuffle	Confiance dans la menace

13.3 Entraînement des modèles

Classification + détection d'anomalies

```
# train.py
import pandas as pd, numpy as np, joblib
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.metrics import classification_report
from sklearn.preprocessing import LabelEncoder
from xgboost import XGBClassifier

df = pd.read_csv("dataset_features.csv")          # features numériques + colonne 'label'
X, y = df.drop(columns=["label"]), df["label"]
le = LabelEncoder(); y_enc = le.fit_transform(y)
Xtr, Xte, ytr, yte = train_test_split(X, y_enc, test_size=0.25, stratify=y_enc, random_state=42)

xgb = XGBClassifier(n_estimators=300, max_depth=6, learning_rate=0.08, subsample=0.9,
    eval_metric="mlogloss")
rf  = RandomForestClassifier(n_estimators=300, class_weight="balanced", random_state=42)
xgb.fit(Xtr, ytr); rf.fit(Xtr, ytr)
for name, m in [("XGBoost", xgb), ("RandomForest", rf)]:
    print(name); print(classification_report(yte, m.predict(Xte), target_names=le.classes_))

# Anomalies : entraîné UNIQUEMENT sur du trafic normal
Xbenign = X[y == "benign"]
iso = IsolationForest(n_estimators=300, contamination="auto", random_state=42).fit(Xbenign)

joblib.dump({"xgb": xgb, "rf": rf, "iso": iso, "le": le, "cols": list(X.columns)}, "models.joblib")
```

Autoencodeur (alternative: Keras/PyTorch quand vous serez à l'aise)

```
# Autoencodeur (sklearn, simple) : erreur de reconstruction = score d'anomalie
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
sc = StandardScaler().fit(Xbenign)
ae = MLPRegressor(hidden_layer_sizes=(16, 6, 16), max_iter=500,
    random_state=42).fit(sc.transform(Xbenign), sc.transform(Xbenign))
def ae_score(x):  # x : DataFrame de features
    z = sc.transform(x); return ((ae.predict(z) - z) ** 2).mean(axis=1)
# Seuil : percentile 99 de l'erreur sur le trafic normal
threshold = np.percentile(ae_score(Xbenign), 99)
joblib.dump({"sc": sc, "ae": ae, "thr": threshold}, "autoencoder.joblib")
```

13.4 Inférence : score, SHAP, clustering, contre-mesure

ml.py (version pédagogique simplifiée)

```
# ml.py — module ML, appelé par l'API ML (ml_api.py)
import os, joblib, numpy as np, pandas as pd, shap, requests

M = joblib.load("models.joblib"); AE = joblib.load("autoencoder.joblib")
explainer = shap.TreeExplainer(M["xgb"])
HOST_CRIT = {"windows-cible": 1.2, "ubuntu-cible": 1.0}     # criticité des hôtes

def featurize(alert, enr):
    r, a = alert.get("rule", {}), alert.get("agent", {})
    row = {c: 0.0 for c in M["cols"]}
    row.update({"rule_level": r.get("level", 0), "firedtimes": r.get("firedtimes", 1),
                "vt_malicious": enr.get("vt_malicious", 0), "misp_hits": enr.get("misp_hits", 0)})
    return pd.DataFrame([row])[M["cols"]]

def countermeasure(cat, alert):
    key = os.getenv("GROQ_API_KEY")
    if not key:
        return {"attaque_reseau": "Bloquer l'IP source au pare-feu, activer fail2ban, vérifier les
            comptes ciblés.",
                "malware": "Isoler la machine, mettre le fichier en quarantaine, vérifier le hash dans
                     MISP.",
                "active_directory": "Désactiver le compte suspect, forcer le changement de mot de
                    passe, revoir les connexions."}.get(cat, "Escalader à un analyste.")
    prompt = (f"Alerte: {alert.get('rule',{}).get('description')} (catégorie {cat}). "
              "Donne 3 actions de réponse prudentes, en une phrase chacune, sans commande
                 destructive.")
    r = requests.post("https://api.groq.com/openai/v1/chat/completions",
        headers={"Authorization": f"Bearer {key}"},
        json={"model": os.getenv("GROQ_MODEL"), "messages": [{"role": "user", "content": prompt}],
            "temperature": 0.2}, timeout=20)
    return r.json()["choices"][0]["message"]["content"]

def analyze_alert(alert, enr):
    X = featurize(alert, enr)
    p = (M["xgb"].predict_proba(X)[0] + M["rf"].predict_proba(X)[0]) / 2
    k = int(np.argmax(p)); cat = M["le"].classes_[k]
    host = alert.get("agent", {}).get("name", "")
    # Score de risque : formule de TRAVAIL (méthode « à définir » dans le README)
    base = 0 if cat == "benign" else float(p[k]) * 70 + min(alert.get("rule", {}).get("level", 0), 15)
         * 2
    risk = round(min(100.0, (base + 10 * enr.get("misp_hits", 0)) * HOST_CRIT.get(host, 1.0)), 1)
    sv = explainer.shap_values(X)                              # explicabilité
    sv_k = sv[k][0] if isinstance(sv, list) else sv[0, :, k]
    top = sorted(zip(M["cols"], sv_k), key=lambda t: -abs(t[1]))[:4]
    Z = AE["sc"].transform(X)
    anomaly = float(((AE["ae"].predict(Z) - Z) ** 2).mean())
    return {"category": cat, "confidence": round(float(p[k]), 3), "risk_score": risk,
            "explanation": [{"feature": f, "impact": round(float(v), 3)} for f, v in top],
            "zero_day_suspect": bool(anomaly > AE["thr"]), "anomaly_score": round(anomaly, 4),
            "countermeasure": countermeasure(cat, alert)}
```

Clustering HDBSCAN (corrélation des alertes)

HDBSCAN

```
# cluster_alerts.py — à exécuter périodiquement (ex. toutes les 5 min) sur les dernières alertes
import hdbscan, pandas as pd
from sklearn.preprocessing import StandardScaler
df = pd.read_csv("recent_alerts_features.csv")
Z = StandardScaler().fit_transform(df[["rule_level", "src_ip_count_5m", "hour", "distinct_ports"]])
df["cluster"] = hdbscan.HDBSCAN(min_cluster_size=5, min_samples=3).fit_predict(Z)   # -1 = bruit
print(df.groupby("cluster").size())   # un cluster = ensemble d'alertes probablement liées
```

Évaluation honnête
Mesurez précision, rappel et **faux positifs** par classe sur des données **non vues** (ex. attaques d'un autre jour). Gardez un jeu de test jamais utilisé à l'entraînement. Un score de 99 % sur le jeu d'entraînement ne prouve rien.

13.5 Exposer le module : l'API ML (échanges JSON avec Shuffle)

Comme dans le schéma (« API ML — JSON »), le module est un **service HTTP à part**, appelé par Shuffle. Il tourne sur le port **8001**, séparé de FastAPI (réponse, port 8000).

ml_api.py

```
# ~/soc/ml/ml_api.py
import os
from typing import Any, Optional
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel
from ml import analyze_alert

app = FastAPI(title="API ML")
TOKEN = os.getenv("ML_API_TOKEN", "change-me")

class AnalyzeReq(BaseModel):
    alert: dict[str, Any]                          # alerte Wazuh (JSON)
    enrichment: Optional[dict[str, Any]] = {}     # résultats Cortex / MISP

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/analyze")
def analyze(req: AnalyzeReq, x_api_token: Optional[str] = Header(None)):
    if x_api_token != TOKEN:
        raise HTTPException(401, "token invalide")
    return analyze_alert(req.alert, req.enrichment or {})
```

Lancer et tester (service systemd à créer comme pour FastAPI, sur le port 8001)

```
cd ~/soc/ml && source .venv/bin/activate
export ML_API_TOKEN='un-autre-token-long' ; uvicorn ml_api:app --host 0.0.0.0 --port 8001
curl -s -X POST http://192.168.57.10:8001/analyze -H "x-api-token: $ML_API_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"alert":{"rule":{"id":"5712","level":10,"description":"sshd brute
      force"},"agent":{"name":"ubuntu-cible"}}}'
```

Réponse JSON attendue : `category`, `confidence`, `risk_score`, `explanation` (SHAP), `zero_day_suspect`, `anomaly_score`, `countermeasure`.

Checklist de validation

- ☐ Le dataset contient au moins quelques dizaines d'exemples par classe.
- ☐ Le rapport de classification est consulté et documenté (précision/rappel par classe).
- ☐ L'API ML (port 8001, /analyze) renvoie catégorie, score, explication SHAP, indicateur d'anomalie et contre-mesure.
- ☐ Les résultats ML apparaissent dans le case TheHive créé par Shuffle.

14. Phase 12 — FastAPI (réponse automatisée)

Dans le README, **FastAPI** est l'API de **réponse automatisée** : elle reçoit une requête **HTTP POST** (envoyée par Shuffle après validation de l'analyste) puis exécute une **commande WinRM** sur la machine Windows. Elle ne fait **pas** d'analyse : l'analyse est faite par l'API ML (section 13).

Principe de sécurité
Une action de réponse peut casser un poste. FastAPI applique donc : authentification par token, **liste blanche d'actions**, champ *approved_by* obligatoire (nom de l'analyste), journalisation de chaque appel.

14.1 Environnement Python

Installation (dossier ~/soc/api)

```
sudo apt -y install python3-venv python3-pip
mkdir -p ~/soc/api && cd ~/soc/api && python3 -m venv .venv && source .venv/bin/activate
pip install fastapi "uvicorn[standard]" pydantic pywinrm python-dotenv
```

14.2 Code de l'API de réponse

main.py (FastAPI : HTTP POST puis WinRM)

```
# ~/soc/api/main.py
import os, time, logging
from typing import Optional
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

logging.basicConfig(filename="/var/log/soc-response.log", level=logging.INFO,
                    format="%(asctime)s %(message)s")
app = FastAPI(title="API de réponse automatisée (FastAPI)")
API_TOKEN = os.getenv("SOC_API_TOKEN", "change-me")

class RespondReq(BaseModel):
    host: str                    # IP ou nom de la machine Windows cible
    action: str                  # "isolate" | "kill_process" | "disable_user"
    approved_by: str             # analyste ayant validé la réponse
    case_id: Optional[str] = None # identifiant du case TheHive
    param: Optional[str] = None   # ex.: nom du compte ou du processus

ALLOWED_ACTIONS = {"isolate", "kill_process", "disable_user"}   # liste blanche

def auth(tok):
    if tok != API_TOKEN:
        raise HTTPException(401, "token invalide")

@app.get("/health")
def health():
    return {"status": "ok", "ts": time.time()}

@app.post("/respond")
def respond(req: RespondReq, x_api_token: Optional[str] = Header(None)):
    auth(x_api_token)
    if req.action not in ALLOWED_ACTIONS:
        raise HTTPException(400, "action non autorisée")
    safe = (req.param or "").replace("'", "").replace(";", "")[:64]   # anti-injection basique
    cmds = {
        "isolate": "New-NetFirewallRule -DisplayName 'SOC-Isolate' -Direction Outbound -Action Block",
        "kill_process": f"Stop-Process -Name '{safe}' -Force -ErrorAction SilentlyContinue",
        "disable_user": f"Disable-LocalUser -Name '{safe}'",
    }
    import winrm                 # commande WinRM vers la machine Windows
    sess = winrm.Session(f"http://{req.host}:5985/wsman",
                         auth=(os.getenv("WINRM_USER"), os.getenv("WINRM_PASS")), transport="ntlm")
    r = sess.run_ps(cmds[req.action])
    logging.info("respond host=%s action=%s param=%s by=%s case=%s rc=%s",
                 req.host, req.action, safe, req.approved_by, req.case_id, r.status_code)
    return {"status": r.status_code, "stdout": r.std_out.decode(errors="ignore"),
            "stderr": r.std_err.decode(errors="ignore"), "approved_by": req.approved_by}
```

14.3 Lancer et tester

Test

```
sudo touch /var/log/soc-response.log && sudo chown $USER /var/log/soc-response.log
export SOC_API_TOKEN='un-token-long-et-aleatoire'
export WINRM_USER='svc_soc' WINRM_PASS='MotDePasseFort!2026'
uvicorn main:app --host 0.0.0.0 --port 8000

# Autre terminal : santé puis test sur le compte de test créé en phase 6
curl -s http://192.168.57.10:8000/health
curl -s -X POST http://192.168.57.10:8000/respond -H "x-api-token: $SOC_API_TOKEN" \
  -H "Content-Type: application/json" \
  -d
      '{"host":"192.168.56.40","action":"disable_user","param":"testsoc","approved_by":"analyste1","case_id":"T-001"}'
```

Documentation interactive : `http://192.168.57.10:8000/docs`.

14.4 Lancer au démarrage (systemd)

Service systemd

```
# /etc/systemd/system/soc-api.service
[Unit]
Description=FastAPI reponse automatisee
After=network.target
[Service]
User=soc
WorkingDirectory=/home/soc/soc/api
EnvironmentFile=/etc/soc-api.env
ExecStart=/home/soc/soc/api/.venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000
Restart=on-failure
[Install]
WantedBy=multi-user.target
```

Activer

```
sudo systemctl daemon-reload && sudo systemctl enable --now soc-api
```

Checklist de validation

- ☐ /health répond « ok ».
- ☐ /respond refuse une action hors liste blanche (400) et un token invalide (401).
- ☐ Un appel valide désactive le compte de test sur Windows (Get-LocalUser le confirme) et la ligne apparaît dans /var/log/soc-response.log.
- ☐ Le nœud HTTP de Shuffle (étape 9 du workflow) déclenche /respond après validation de l'analyste.

15. Phase 13 — SOC Copilot (LLM + RAG)

Le **SOC Copilot** est l'assistant de l'analyste : il résume les alertes, explique les logs en langage simple et guide l'enquête. Dans le schéma, il lit les données du **Wazuh indexer** (RAG) et s'appuie sur un LLM local ou sur l'API Claude.

15.1 Principe du RAG

1. Question de l'analyste : « Que s'est-il passé sur ubuntu-cible depuis 10 h ? »
2. Le Copilot interroge l'indexer pour récupérer les alertes pertinentes (filtre agent + temps).
3. Il construit un prompt contenant ces alertes (nettoyées, tronquées) et la question.
4. Le LLM répond en s'appuyant **uniquement** sur ces données, avec des références (rule.id, horodatage).

15.2 Choix du LLM

Option	Avantages	Limites
LLM local (ex. Ollama + un modèle open source)	Données sensibles ne quittent pas le labo ; gratuit à l'usage	Demande RAM/GPU ; qualité variable selon la taille du modèle
API Claude (Anthropic)	Très bonne qualité d'analyse et de rédaction ; simple à intégrer	Les données partent vers un service externe : anonymiser IP/noms si nécessaire

15.3 Squelette du Copilot

Copilot avec API Claude (version minimale)

```
# copilot.py
import os, requests, anthropic
IDX = "https://192.168.57.10:9200/wazuh-alerts-*/_search"
AUTH = ("admin", os.getenv("WAZUH_IDX_PASS"))

def fetch_alerts(agent=None, minutes=60, min_level=5, size=40):
    must = [{"range": {"timestamp": {"gte": f"now-{minutes}m"}}},
            {"range": {"rule.level": {"gte": min_level}}}]
    if agent: must.append({"match": {"agent.name": agent}})
    q = {"size": size, "sort": [{"rule.level": "desc"}], "query": {"bool": {"must": must}}}
    hits = requests.get(IDX, auth=AUTH, json=q, verify=False).json()["hits"]["hits"]
    return [{"t": h["_source"]["timestamp"], "agent": h["_source"]["agent"]["name"],
             "rule": h["_source"]["rule"]["id"], "lvl": h["_source"]["rule"]["level"],
             "desc": h["_source"]["rule"]["description"]} for h in hits]

SYSTEM = ("Tu es un assistant pour analyste SOC débutant. Réponds en français, "
          "de façon courte et claire. "
          "Base-toi UNIQUEMENT sur les alertes fournies ; si l'information manque, dis-le. "
          "Cite rule.id et heure. Propose des étapes d'investigation, "
          "jamais d'action destructive.")

def ask(question, agent=None):
    ctx = fetch_alerts(agent)
    client = anthropic.Anthropic()                    # clé dans la variable ANTHROPIC_API_KEY
    prompt = f"Alertes (JSON): {ctx}\n\nQuestion: {question}"
    msg = client.messages.create(model=os.getenv("COPILOT_MODEL", "claude-sonnet-5-5"),
        max_tokens=700, system=SYSTEM, messages=[{"role": "user", "content": prompt}])
    return msg.content[0].text

if __name__ == "__main__":
    print(ask("Résume les alertes critiques de la dernière heure.", agent="ubuntu-cible"))
```

Bonnes pratiques de sécurité pour le Copilot
Les logs peuvent contenir du texte contrôlé par l'attaquant (ex. un nom d'utilisateur contenant des instructions). Traitez-les comme des **données**, jamais comme des ordres : consigne système explicite, aucune action automatique déclenchée par la sortie du LLM, et le Copilot reste en **lecture seule** sur l'indexer. Utilisez un compte indexer à droits limités.

Checklist de validation

- ☐ Le Copilot répond à une question simple en citant des alertes réelles.
- ☐ Il dit « je ne sais pas » quand aucune alerte ne correspond.
- ☐ Les clés API sont dans des variables d'environnement, jamais dans le code ni sur Git.

16. Phase 14 — Scénarios de bout en bout et démonstration

Cette phase valide que **tout le pipeline** fonctionne ensemble. Pour chaque scénario, suivez la chaîne complète et capturez des écrans à chaque étape.

16.1 Scénario A — Brute force SSH

1. **Kali** : `hydra -l testsoc -P petite_liste.txt ssh://192.168.56.30 -t 4` (liste courte !).
2. **Suricata/Wazuh** : alertes d'échecs d'authentification puis règle de brute force (niveau ≥ 10).
3. **Shuffle** : workflow déclenché ; IP 10.10.10.20 enrichie (Cortex/MISP/VT — IP privée, donc peu de résultats : c'est normal en labo).
4. **API ML** (appelée par Shuffle) : catégorie « attaque_reseau », score élevé, SHAP : fréquence des échecs + niveau de règle.
5. **TheHive** : case créé avec observables, score, explication et contre-mesure.
6. **Analyste** : consulte le Copilot, valide le blocage de l'IP dans pfSense, clôture le case avec un commentaire.

16.2 Scénario B — Fichier malveillant (EICAR) sur Ubuntu

1. Dépôt du fichier EICAR dans /tmp ; ClamAV le détecte au scan planifié ; Wazuh remonte l'alerte.
2. Shuffle extrait le **SHA256** ; Cortex (VirusTotal) et MISP confirment ; l'API ML classe « malware » ; TheHive crée le case.

16.3 Scénario C — Abus de compte sur Windows (famille Active Directory)

1. Création d'un compte local, ajout au groupe Administrateurs et échecs de connexion répétés ; les journaux Security et Sysmon les enregistrent.
2. Règle locale 100100 déclenchée ; API ML : « active_directory » ; le case propose la désactivation du compte.
3. L'analyste valide : Shuffle appelle FastAPI `/respond` (HTTP POST) qui exécute *disable_user* par WinRM ; le résultat est ajouté au case.

16.4 Scénario D — Scan réseau

1. `nmap -sS -sV -p- 192.168.56.0/24` depuis Kali : alertes Suricata ; l'API ML (HDBSCAN) regroupe des dizaines d'alertes en **un** cluster (un seul incident).

16.5 Indicateurs à mesurer pour le rapport

Indicateur	Définition	Objectif de labo
Taux de détection	Attaques simulées détectées / attaques lancées	≥ 90 %
Faux positifs	Alertes bénignes classées malveillantes	Le plus bas possible ; à documenter
MTTD (délai de détection)	Attaque → alerte Wazuh	< 1 minute
MTTR simulé	Alerte → case TheHive créé	< 2 minutes (automatisé)
Réduction du bruit	Alertes brutes / nombre de clusters	Facteur ≥ 5

Plan de démonstration (15 min) : 1) présenter l'architecture (2 min) ; 2) lancer le scénario A en direct (5 min) ; 3) montrer le case TheHive avec score/SHAP (3 min) ; 4) poser une question au

Copilot (2 min) ; 5) montrer les indicateurs (3 min).

Checklist de validation

- ☐ Les quatre scénarios passent de bout en bout au moins une fois.
- ☐ Vous avez des captures d'écran et un tableau d'indicateurs.
- ☐ Un snapshot final « demo-ok » de chaque VM est pris.

17. Dépannage : problèmes fréquents

Symptôme	Causes probables	Solutions
Agent Wazuh « Disconnected » / jamais visible	Pare-feu pfSense, mauvaise IP manager, heure différente	Tester `nc -vz 192.168.57.10 1514` ; contrôler règles pfSense ; lire /var/ossec/logs/ossec.log ; synchroniser l'heure
Dashboard Wazuh très lent / indexer s'arrête	RAM insuffisante	Augmenter la RAM de la VM SOC ; réduire le heap de l'indexer ; désactiver temporairement MISP/Shuffle
Pas d'alertes Suricata dans Wazuh	Syslog non autorisé, mauvais format, mauvaise IP source	Vérifier le bloc <remote>, `tcpdump -ni any udp port 514` sur le SOC, activer le log all pour debug
TheHive ne démarre pas	Cassandra/Elasticsearch pas prêts, mémoire	`docker compose logs` ; attendre 2–3 min ; vérifier vm.max_map_count
Conflit de port 443 / 9200	Wazuh et MISP/Elasticsearch sur le même hôte	Publier MISP sur 8443 ; ne pas exposer l'ES des conteneurs
Webhook Shuffle ne reçoit rien	Niveau < 7, URL erronée, Shuffle non joignable	Tester avec curl vers l'URL du hook ; consulter /var/ossec/logs/integrations.log
Quota VirusTotal dépassé	Offre gratuite limitée	Ajouter une pause, un cache par hash, ou limiter aux hash/IP publiques
WinRM échoue (401 / timeout)	Auth, pare-feu Windows, profil réseau Public	`winrm quickconfig` ; profil réseau Privé ; règle entrante 5985 depuis 192.168.57.10
Modèle ML donne toujours la même classe	Dataset déséquilibré ou trop petit	Collecter plus de données, *class_weight*, vérifier les étiquettes

18. Sécurisation de la plateforme elle-même

Un SOC est une cible de choix : sécurisez-le comme un environnement réel.

- **Mots de passe et clés API** : changez tous les comptes par défaut ; stockez les secrets dans un gestionnaire (ou fichier d'environnement en 600), jamais dans Git.
- **Segmentation** : le SOC n'est pas joignable depuis Kali ni depuis Internet ; accès d'administration limité à votre poste.
- **TLS** : activez HTTPS sur FastAPI (reverse proxy Nginx) et remplacez les certificats auto-signés si possible.
- **Moindres privilèges** : comptes de service dédiés (Shuffle→TheHive, FastAPI→WinRM, Copilot→indexer lecture seule).
- **Actions de réponse** : liste blanche, validation humaine, journalisation de chaque action.
- **Mises à jour et sauvegardes** : snapshots avant mise à jour ; exportez régulièrement les règles locales, workflows Shuffle et modèles ML.
- **Journalisation de la plateforme** : surveillez aussi les logs du serveur SOC avec un agent Wazuh dédié.

19. Planning proposé et ordre des étapes

Semaine	Étapes	Livrable
1	Hyperviseur, réseaux, VM, pfSense, serveur SOC + Wazuh	Dashboard Wazuh accessible
2	Agents Ubuntu + Windows (Sysmon), ClamAV, Suricata, Kali, tests Wazuh seuls	Journal des tests de détection
3	TheHive + Cortex, puis MISP + VirusTotal	Cases de test avec enrichissement
4	Shuffle + intégration Wazuh, workflow Cortex/MISP → TheHive	Alerte → case TheHive automatique
5	Dataset, entraînement ML, SHAP, HDBSCAN, anomalies, API ML, score de risque	Modèles + API ML + rapport d'évaluation
6	FastAPI + WinRM (réponse), SOC Copilot, scénarios de bout en bout, indicateurs	Démo + rapport final

Ordre exact des 16 étapes : (1) machines et réseau → (2) serveur SOC → (3) Wazuh → (4) agent Ubuntu → (5) agent + Sysmon Windows → (6) test des remontées → (7) Kali et attaques → (8) TheHive → (9) Cortex → (10) MISP → (11) VirusTotal → (12) Shuffle → (13) module ML et API ML → (14) FastAPI (réponse automatisée) → (15) Copilot SOC → (16) tests et démo. pfSense/Suricata s'insèrent après l'étape 6 (ou dès l'étape 1 si vous maîtrisez la virtualisation réseau).

19.1 Les 4 questions à poser pour chaque outil

1. Pourquoi je l'installe ?
2. Où je l'installe ?
3. Que doit-il recevoir ?
4. Comment je vérifie que ça marche ?

19.2 Bonnes habitudes

- Avancez petit à petit ; validez une phase avant la suivante.
- Notez chaque commande et chaque erreur dans un carnet.
- Faites des captures d'écran (utiles pour le rapport et la démo).
- Utilisez les snapshots de VM systématiquement.

20. Glossaire

Terme	Définition simple
SOC	Centre d'opérations de sécurité : équipe + outils qui surveillent et réagissent aux menaces
SIEM	Outil qui centralise et corrèle les logs (ici Wazuh)
SOAR	Plateforme d'automatisation des réponses (ici Shuffle)
IDS / IPS	Système de détection / de prévention d'intrusion (Suricata)
IOC	Indicateur de compromission : IP, domaine, hash, URL malveillants
FIM	File Integrity Monitoring : surveillance des modifications de fichiers
Observable	Élément analysable d'un incident (IP, hash, domaine…) dans TheHive
Sysmon	Outil Microsoft qui journalise finement l'activité Windows
EVE JSON	Format de logs JSON de Suricata
MITRE ATT&CK	Référentiel des techniques d'attaque, utilisé par les règles Wazuh
SHAP	Méthode qui explique quelles variables ont influencé une prédiction
HDBSCAN	Algorithme de clustering qui regroupe des éléments similaires sans fixer le nombre de groupes
Isolation Forest / Autoencodeur	Modèles de détection d'anomalies (comportements inhabituels)
RAG	Le LLM répond en s'appuyant sur des documents/données récupérés (ici les alertes)
WinRM	Gestion à distance de Windows (utilisée pour la réponse active)
MTTD / MTTR	Délai moyen de détection / de réponse

21. Documentation officielle à consulter

Les numéros de version et certaines options changent régulièrement. Pour toute commande d'installation, vérifiez la page officielle correspondante :

- Wazuh : documentation.wazuh.com (Installation assistant, Agents, Integrations → Shuffle)
- pfSense / Suricata : docs.netgate.com
- TheHive et Cortex : docs.strangebee.com et github.com/TheHive-Project/Cortex
- MISP : misp-project.org et github.com/MISP/misp-docker
- Shuffle : shuffler.io/docs et github.com/Shuffle/Shuffle
- ClamAV : docs.clamav.net
- Sysmon : learn.microsoft.com (Sysinternals — Sysmon)
- FastAPI : fastapi.tiangolo.com
- XGBoost, scikit-learn, SHAP, HDBSCAN : documentations officielles de chaque bibliothèque
- Groq : console.groq.com/docs ; Claude API : docs.claude.com

Pour aller plus loin
Quand le pipeline fonctionne, améliorez-le : plus de scénarios d'attaque (MITRE ATT&CK), règles Wazuh personnalisées, réentraînement régulier du ML avec les décisions de l'analyste, et tableau de bord de suivi des indicateurs.